

MATH3881/5881: Statistical Machine Learning Theory

Week 5: Supplement: Backpropagation in CNNs

Sarat Moka

UNSW, Sydney

Optional reading. This supplement is optional material for students who want the explicit gradient expressions for a convolutional layer and the full backprop sweep through a CNN. These same gradients are computed automatically by any modern autodiff framework (PyTorch, TensorFlow, JAX); the value of working through them by hand is to see exactly what each layer's gradient operation looks like, and to recognise that the backward pass through a convolutional layer is itself a convolution. This material is not examinable.

Training a CNN requires computing gradients of the loss $C(\theta)$ with respect to all kernel weights and biases. The framework is exactly the reverse-mode autodiff / backpropagation from Week 3, applied to the convolutional layer $a = S(W \star x + b)$ (Section 4 of the Week 5 lecture notes). We derive the three key gradient expressions and then assemble them into a full CNN backward pass.

Setup

Consider a single convolutional layer with kernel $W \in \mathbb{R}^{K_h \times K_v}$, bias b , input $x \in \mathbb{R}^{M_h \times M_v}$, and output $z = W \star x + b \in \mathbb{R}^{M'_h \times M'_v}$ (where $M'_h = M_h - K_h + 1$, $M'_v = M_v - K_v + 1$, ignoring padding for brevity). The activation $a = S(z)$ is passed to subsequent layers. Suppose through the rest of the network and loss function, we have already computed the upstream gradient

$$\delta \equiv \frac{\partial C}{\partial z} \in \mathbb{R}^{M'_h \times M'_v}$$

by reverse-mode autodiff on the later layers.

Gradient with Respect to the Kernel

The gradient of C with respect to each kernel entry $w_{k,l}$ accumulates contributions from every output $z_{i',j'}$ that $w_{k,l}$ affects:

$$\frac{\partial z_{i',j'}}{\partial w_{k,l}} = x_{i'+K_h-1-k, j'+K_v-1-l}.$$

Summing over all output positions by the chain rule:

$$\frac{\partial C}{\partial w_{k,l}} = \sum_{i'=1}^{M'_h} \sum_{j'=1}^{M'_v} \delta_{i',j'} x_{i'+K_h-1-k, j'+K_v-1-l}. \quad (5.1)$$

This is precisely the cross-correlation of x with δ , i.e., $\nabla_W C = x \star \delta$ (with δ as the kernel). In full matrix form:

$$\nabla_W C = x \tilde{\star} \delta, \quad (5.2)$$

where $\tilde{\star}$ denotes cross-correlation (convolution without flipping). The gradient of the bias is simply $\nabla_b C = \sum_{i',j'} \delta_{i',j'}$.

Gradient with Respect to the Input

The gradient with respect to the input $x_{i,j}$ accumulates contributions from all output positions (i',j') such that (i,j) falls within the kernel window at (i',j') :

$$\frac{\partial z_{i',j'}}{\partial x_{i,j}} = w_{K_h-1-(i-i'), K_v-1-(j-j')},$$

when $0 \leq i - i' \leq K_h - 1$ and $0 \leq j - j' \leq K_v - 1$. Summing:

$$\frac{\partial C}{\partial x_{i,j}} = \sum_{i',j'} \delta_{i',j'} w_{K_h-1-(i-i'), K_v-1-(j-j')}. \quad (5.3)$$

This equals the **full convolution** of δ (zero-padded to $(M_h + K_h - 1) \times (M_v + K_v - 1)$) with the **flipped kernel** W^{flip} where $[W^{\text{flip}}]_{k,l} = w_{K_h-1-k, K_v-1-l}$:

$$\nabla_x C = W^{\text{flip}} \star_{\text{full}} \delta. \quad (5.4)$$

This gradient propagates to the previous layer as its δ , enabling backprop to continue.

Remark 5.0.1

The *transpose convolution* (sometimes called *deconvolution*) used in generative architectures and segmentation networks is exactly the gradient operation (5.4): it maps from the output space back to the input space using the flipped kernel. This is the adjoint of the forward convolution map, viewed as a linear operator.

Gradient Through Pooling

For **max-pooling**, only the location that achieved the maximum in the forward pass receives gradient from the upstream δ ; all other positions in the window receive zero gradient. Formally, for the window $\mathcal{I}_{(i,k)}$ with maximiser (i_*, k_*) :

$$\frac{\partial C}{\partial [a_{(j)}^{[\ell-1]}]_{i',k'}} = \begin{cases} \delta_{(j),i,k} & \text{if } (i', k') = (i_*, k_*), \\ 0 & \text{otherwise.} \end{cases} \quad (5.5)$$

The forward pass must therefore cache the argmax locations to use during backpropagation. For **average-pooling**, the upstream gradient is distributed uniformly: each input position in the window receives $\delta_{(j),i,k} / |\mathcal{I}_{(i,k)}|$.

Full Backpropagation in a CNN

Combining the above, a single pass of backprop through a CNN (connecting to the autodiff graph of Week 3) proceeds:

1. **Forward pass:** compute and cache $z^{[\ell]}$, $a^{[\ell]}$, and max-pool argmax locations for all layers.
2. **Output layer:** compute $\delta^{[L]} = \nabla_{a^{[L]}} C$ from the loss (e.g. cross-entropy gradient).
3. **Dense layers:** apply standard FNN backprop from Week 2.
4. **Flatten layer:** reshape the gradient vector back to tensor form.
5. **Pooling layer:** upsample δ to the input size using (5.5).
6. **Convolutional layer:** (a) compute $\nabla_{W^{[\ell]}} C$ via (5.2); (b) compute $\delta^{[\ell-1]} = \nabla_{x^{[\ell]}} C$ via (5.4) and pass to the previous layer.
7. Repeat (5),(6) for all conv/pool layers; accumulate $\nabla_{W^{[\ell]}} C$ for each trainable layer.
8. **Optimisation step:** update all $W^{[\ell]}$ using any method from Week 4 (SGD, Adam, etc.).